

Content Health Manager — SPFx Submission Test & Documentation Guide

Prepared for the Microsoft SharePoint SPFx Submission Test team, to support Microsoft 365/AppSource Marketplace validation of this solution against the SPFx solution store rules.

1. Solution identity

Item	Value
Solution / package name	Content Health Manager (<code>centralized-content-health-manager</code> , generator solution name <code>sp-content-health-manager</code>)
Package file	<code>solution/dev-sky-content-health-manager.sppkg</code>
Solution ID	<code>61faf8a2-2f3f-49f0-993e-fa1db6c70fb2</code>
Solution version	<code>1.0.0.0</code> (<code>config/package-solution.json</code>)
Feature ID	<code>ac57c893-8be0-4d4e-9b78-7367235289b2</code> — "Centralized Content Health Manager"
Web part component ID	<code>96612611-2584-411f-bcf3-2c19a26eb6bc</code>
Web part alias	<code>ContentHealthManagerWebPart</code>
SPFx framework version	<code>1.22.1</code> (see <code>devDependencies</code> in <code>package.json</code> ; the Yeoman scaffold record in <code>.yo-rc.json</code> and the README badge still show the original <code>1.21.1</code> generator version — cosmetic only, the shipped bundle targets 1.22.1)
Node engine required to build	<code>>=22.14.0 <23.0.0</code>
Publisher	<code>dev-sky.net</code> , MPN ID <code>6499491</code>
Privacy / Terms URLs	<code>https://www.dev-sky.net/PrivacyPolicy/app-privacy.html / .../app-terms.html</code>
Categories	Content management, Productivity, Site Design
Supported locales	<code>en-US</code> , <code>de-DE</code>
Supported hosts	<code>SharePointWebPart</code> , <code>SharePointFullPage</code> (Teams/Outlook/Office are not declared — see §7)
<code>requiresCustomScript</code>	<code>false</code>
<code>isDomainIsolated</code>	<code>false</code>
<code>skipFeatureDeployment</code>	<code>true</code> (tenant-wide availability once added to the app catalog; no per-site feature activation step)
<code>supportsThemeVariants</code> / <code>supportsFullBleed</code>	<code>true</code> / <code>true</code>

2. What the solution does

Content Health Manager is a single web part for SharePoint site owners, content managers, and IT administrators. After the user picks one or more site collections, it gives a consolidated audit view of a selected site's pages and document libraries:

- Scans page content for **broken hyperlinks**.
- Flags pages with an **unpublished draft** awaiting approval.
- Flags pages, libraries, and items with **unique (non-inherited) permissions**.
- Shows which pages/files are **checked out**, and to whom.
- Finds library/list items **not modified since a given date** ("old content").
- Looks up the **effective permissions** of a specific user or SharePoint/Entra group on a page or library, including nested group membership.
- Looks up who holds a **built-in Entra ID directory role** (e.g. Global Administrator) tenant-wide.

The web part is strictly **read-only** against site content and permissions — see §5.

3. Permissions requested and why

Permission	Type	Where requested	Purpose	Admin action required
<code>RoleManagement.Read.Directory</code>	Microsoft Graph, delegated	<code>config/package-solution.json</code> → <code>webApiPermissionRequests</code>	Powers the " Built-in Entra ID Roles " tab of the Permissions dialog: resolves the members of a built-in directory role via Graph's <code>/directoryRoles</code> endpoint.	Yes. Tenant admin must approve this pending request in SharePoint Admin Center → Advanced → API access before that one tab works.

All other Graph and SharePoint REST calls the web part makes (reading sites, pages, lists, items, role assignments, group membership, effective permissions) run under the SharePoint Online client's already-consented Graph scopes and the signed-in user's own SharePoint permissions — no separate permission request is needed for them, and the app cannot see or do anything the signed-in user could not already do through the browser.

What the test team should expect before admin approval: attempting to use the Entra Roles tab without approval surfaces a clear in-product message explaining that the tenant admin has not yet approved the permission, rather than a silent failure or a raw error. A role that exists but has never been assigned to anyone in the tenant also resolves to "no one holds this role" rather than an error — this is expected, not a defect.

4. Data read by the solution (no write access)

API surface	Endpoints used	Purpose
Microsoft Graph v1.0	<code>/sites/{id}/pages,</code> <code>/sites/{id}/pages/{id}/microsoft.graph.sitePage,</code> <code>/sites/{id}/lists, /sites/{id}/lists/{id}/items, /directoryRoles,</code> <code>/directoryRoles/{id}/members, /groups/{id}/transitiveMembers,</code> <code>/groups/{id}/members</code>	Page/library enumeration, page content (canvas layout, for link extraction), old-content queries, directory role and Entra group membership resolution
SharePoint REST	<code>_api/web/lists, _api/web/lists('{id}')/items,</code> <code>_api/web/lists('{id}')/roleassignments,</code> <code>_api/web/getusereffectivepermissions,</code> <code>_api/web/sitegroups/getbyid,</code> <code>_api/web/GetFileByServerRelativeUrl(...)/ListItemAllFields,</code> <code>_api/web/ensureuser</code>	Lists/libraries metadata, checked-out items, direct role assignments, effective-permission bitmask lookups, SharePoint group membership, resolving a user's login name for the permission check UI

All of the above are **GET** requests except `_api/web/ensureuser` (a **POST** that only resolves/normalizes a user's login name so it can be looked up — it does not create or change any content or permission) and one classic-REST list-item query that uses **POST** purely to carry a CAML `ViewXml` query payload (also read-only). **No **PATCH**, **PUT**, or **DELETE** calls exist anywhere in the codebase.** This can be spot-checked by the test team via the browser Network tab while exercising every feature — expect only **GET** calls plus the two read-only **POST** calls above.

5. External (non-Microsoft) network calls — please read before flagging

The broken-link scanner (`src/Core/PageProcessing.ts`) issues a client-side `fetch(url, { method: 'HEAD', mode: 'no-cors' })` for every hyperlink URL found in a scanned page's content, in order to determine whether that link resolves. These requests:

- Run entirely in the end user's browser session, using the browser's own network stack — equivalent to the user opening the link themselves.
- Can target **any domain** referenced by a link on the scanned page, including non-Microsoft domains, since that is the point of a broken-link checker.
- Send no cookies, tokens, or SharePoint data to the target URL beyond a bare HEAD request (`mode: 'no-cors'` also means the response body/headers are not readable by the app — only whether the request succeeded).
- Are not proxied through, logged by, or reported to any dev-sky.net service. Results (reachable/broken) stay in the browser's in-memory state for that session only.

This is the only place the solution talks to a domain other than the tenant's own SharePoint/Graph endpoints. Flagging it as expected behavior up front should save a review round-trip.

6. Architecture & technology stack

- React 17 function/class components, Fluent UI React v8 ([@fluentui/react](#)) and Fluent UI v9 ([@fluentui/react-components](#)) side by side, wrapped in [FluentProvider](#) (theme: [teamsLightTheme](#) / [teamsDarkTheme](#) based on the SharePoint theme).
- [IdPrefixProvider](#) (`value="msz"`) scopes Fluent v9's generated element IDs, so multiple instances of the web part on the same page do not collide — verified by placing two instances on one page and confirming both function independently.
- PnP [@pnp/spfx-controls-react](#) v3.22.0 controls: [SitePicker](#), [ListView](#), [PeoplePicker](#), [FieldDateRenderer/FieldTextRenderer](#), [WebPartTitle](#).
- [react-resizable-panels](#) for the resizable split view inside the Permissions dialog (principal list / member tree).
- No [externals](#) are declared in [config/config.json](#) and no third-party CDN scripts are loaded — all JS/CSS ships inside the [.sppkg](#).
- Theme integration: [onThemeChanged](#) maps the SharePoint theme's semantic colors/palette onto CSS custom properties consumed by [ContentHealthManager.module.scss](#) and [WebPartTitleOverrides.global.scss](#).

7. Known limitations / things to note, not defects

- **Teams/Outlook/Office context code with no matching host declaration:** [ContentHealthManagerWebPart.ts](#) contains environment-detection logic for Teams, Office, and Outlook hosts ([_getEnvironmentMessage](#)), but [supportedHosts](#) in the manifest only lists [SharePointWebPart](#) and [SharePointFullPage](#). The web part is **not** installable as a Teams tab, Office add-in, or Outlook add-in — that branch of code is inert in this build. Nothing to test there.
- **Permission model:** viewing detailed role assignments on a site requires the signed-in **test account to have Manage Permissions rights** on that site; a lower-privileged account will see reduced or no data in the Permissions dialog by design (not a bug).
- The npm package version ([package.json](#) → `"version": "0.0.1"`) is unrelated to the published solution version; the SPFx solution version that governs the [.sppkg](#) is [1.0.0.0](#) in [config/package-solution.json](#).
- [README.md](#) at the repo root still has generic Yeoman-scaffold placeholder text (version history table, "topic 1/2/3", etc.) — not used for anything the test team evaluates; the authoritative short/long descriptions are in [config/package-solution.json](#) (§1).

8. Test tenant prerequisites

To exercise every feature, prepare one SharePoint Online test site with:

1. Several modern pages with hyperlinks — at least one page with a working link and one with a deliberately broken link (e.g. a link to a deleted page or a non-existent path), to validate the broken/OK classification.
2. At least one page saved as a **minor version / unpublished draft**, to validate the "Needs approval" flag.
3. At least one file or page **checked out** to a user, to validate the "Checked out" flag and the checked-out-items query.
4. At least one library or page with **broken permission inheritance** (unique permissions), to validate the "Unique permissions" flag and the Permissions dialog's role-assignment view.
5. A SharePoint group and an Entra ID security group, both granted access somewhere in the site, with at least one nested group inside each — to validate group member/nested-group resolution in the Permissions dialog's tree view.

6. Two test user accounts: one with access to the item under test, one without — to validate the "search a user or group" effective-access lookup (**Has access — Permission level: ...** vs. **does not have access**).
7. A tenant admin who has approved the **RoleManagement.Read.Directory** API permission request (§3), to validate the Entra ID Roles tab end-to-end; also try it once **before** approval to confirm the explanatory pending-approval message.
8. The test account used for the walkthrough should have **Manage Permissions** rights on the test site (see §7).

9. Feature walkthrough / test scenarios

9.1 Site selection

1. Add the web part to a modern page.
2. In **Select sites**, search and pick one or more site collections. *Expected:* a filterable multi-select site picker.
3. In **Choose a Site**, pick one of the selected sites to analyze. *Expected:* the two analysis tabs ("Page Library", "Library Analysis") become available.

9.2 Page Library tab — broken link analysis

1. Select the **Page Library** tab. *Expected:* a list of the site's pages loads.
2. Click **Find broken links**. *Expected:* every page is scanned; per-page broken-link counts populate.
3. Click **Process page** on a single page. *Expected:* only that page is (re-)scanned.
4. Click **Load details** on a page (or use "Load details for every page"). *Expected:* approval status, unique-permission flag, and checked-out-by populate for the page row(s).
5. Click **Open details** on a page. *Expected:* a **Page report** dialog opens showing total links, broken links, a "show only broken links" toggle, and a per-link "Show Content" toggle that reveals the raw HTML context the link was found in.
6. Click **Permissions** on a page row. *Expected:* the Permissions dialog opens for that page (§9.4).

9.3 Library Analysis tab

1. Select the **Library Analysis** tab.
2. Toggle **Libraries** / **Lists** to include/exclude each in the scope. *Expected:* the library/list dropdown filters accordingly.
3. Pick a **date**. *Expected:* the date gates both the "old content" queries and the checked-out-items query.
4. Click **Query all libraries**. *Expected:* every library/list in scope is scanned for items not modified since the selected date.
5. Click **Query library** with one specific library/list selected. *Expected:* only that library/list is scanned.
6. Click **Checked-out items**. *Expected:* items currently checked out anywhere in scope are listed, with who has them checked out.
7. Click **Open details** on a library. *Expected:* a **Library report** dialog opens with library metadata (template, description, item count, created/modified dates, versioning/attachment/folder-creation settings) plus the matched item list and totals.
8. Click **Permissions** on a library row (or **Show Permissions** inside the library report). *Expected:* the Permissions dialog opens for that library (§9.4).

9.4 Permissions dialog

1. **Permissions tab:** shows the resolved role assignments (Name, Type, Login name, Roles) for the selected page/library/item, walking up to the nearest object that actually owns unique permissions if the item inherits. *Expected:* selecting a group row expands a tree of its members (and, for SharePoint groups, nested Entra groups) in the resizable side panel.

2. Use **Search for a user or group...** to look up a specific principal. *Expected:* result shows either `<name> has access — Permission level: <level>` or `<name> does not have access to this item.`
3. **Built-in Entra ID Roles tab:** pick a role (e.g. Global Administrator) from **Built-in Entra ID role**. *Expected (after admin approval, §3):* the members holding that role tenant-wide are listed, or a "No one in this tenant is currently assigned the {role} role" message if none. *Expected (before admin approval):* a message explaining the pending permission approval, not a raw error.

9.5 Multi-instance / theming sanity check

1. Add two instances of the web part to the same page. *Expected:* both operate independently with no ID collisions or state bleed.
2. Toggle the site's theme (light/dark or a custom theme) and open the page again. *Expected:* the web part's colors follow the SharePoint theme; check both Fluent v8 and v9 controls pick it up.

9.6 Property pane

1. Open the web part's edit panel. *Expected:* a **Description** field is present under a single group (satisfies the store requirement that a web part expose configurable properties). The web part title is editable directly on the canvas via the title control, independent of the property pane.

10. Version history

Version	Notes
1.0.0.0	Initial Marketplace submission — broken-link analysis, library/list old-content and checked-out-item queries, unique-permission detection, role-assignment and effective-permission lookup, built-in Entra ID role lookup.